

Team Task Split

Person A (experienced) · Person B (newer to this stack)

Why the split is structured this way

The riskiest, most judgment-heavy work — data fusion, the risk-scoring formula, LLM prompt design — goes to Person A, because mistakes there are subtle and hard to debug under time pressure. The frontend track given to Person B has a well-worn path (map libraries have huge tutorials/communities) and fails loudly and visibly when something's wrong, which makes it a better fit for someone newer to the stack. This is not about one role being less important — the map/timeline IS the demo. Judges watch the screen, not the backend.

Person A (Pro) — Data, Backend, Risk Engine, LLM Integration

Owns:

- IBTrACS track ingestion and cleaning
- GEE integration (elevation, land cover pulls)
- Overpass API integration for infra points
- The risk-scoring formula — design, tune, and be ready to defend it in Q&A
- FastAPI backend and the GeoJSON output contract
- Gemini 3.7 Flash prompt design and integration
- Final data-quality sanity checks against the real historical storm's known impact

Definition of done for Day 1:

A working endpoint that returns a GeoJSON risk surface for at least 3-4 timesteps of the chosen storm, using real elevation/infra/track data (placeholder risk weights are fine, they get tuned Day 2).

Person B (Newer to the stack) — Frontend, Map, Timeline, Polish

Owns:

- Project scaffold (Vite or Next.js) and MapLibre GL JS setup
- The storm-track animation and timeline slider — THE key demo visual
- Layer toggles (risk heatmap, roads, hospitals, power)
- Styling and visual polish so the map looks deliberate, not default
- Deployment (Vercel/Render) and the fallback demo recording

Minimum you need to learn — in order, don't over-invest:

- 1. What GeoJSON looks like (it's just JSON with a specific 'features' array shape — 20 min read is enough).
- 2. MapLibre GL JS quickstart: adding a map, adding a GeoJSON source, adding a layer that colors features by a property (this is literally your risk heatmap).
- 3. A basic HTML range slider bound to state, driving which timestep's GeoJSON is currently shown on the map.

- That's it — you do not need to learn backend, GEE, or prompt engineering to contribute meaningfully here.

Definition of done for Day 1:

A map that loads, shows a timeline slider, and renders placeholder GeoJSON points — ready to swap in Person A's real data on Day 2 morning without restructuring anything.

Sync points (do not skip these)

- End of Day 1 morning: agree on the exact GeoJSON field names before either of you builds further against them.
- End of Day 1: confirm the data contract actually works end-to-end with at least dummy data.
- Day 2 midday: full integration test together, not separately.
- Day 2 evening: both present the pitch to each other once before the real judging, cold.

Shared responsibility — both people

- Both should be able to explain the risk-scoring formula and answer 'is this RAG' correctly in Q&A (it's not — see the Architecture doc). Judges sometimes quiz whichever teammate is standing closest.
- Both should know the fallback plan if GEE, Gemini, or wifi fails live during judging.